

FIAP

NABA

Apache
Spark





- Plataforma de computação em Cluster rápida, tolerante a falhas e de propósito geral
- 100x mais rápido que Mapreduce em memória
- 10x mais rápido que Mapreduce em disco
- Desenvolvido em Scala
- Aplicações em Java, Scala, Python e R
- Bibliotecas para SQL, Streaming, Machine Learning e grafos



- Processamento em larga escala
- Cluster, StandAlone, Docker ou Cloud
- Projeto Apache top-level
- Maior cluster conhecido possui 8000 nós
- Muito utilizado para ETL e análise de dados
- Processamento de PBs de dados
- Framework mais utilizado para processamento e análise de dados atualmente



- Começou em 2009 como um projeto de pesquisa no UC Berkeley – AMPLab
- Os pesquisadores observaram que o MapReduce era ineficiente para empregos de computação iterativa e interativa
- Logo após a sua criação em 2009, já era 10-20 vezes mais rápido do que o MapReduce em alguns casos
- Além da UC Berkeley, os principais contribuidores incluem Databricks, Yahoo ! e Intel
- Foi aberto em março de 2010 e foi transferida para a ASF

Componentes

Spark SQL and
DataFrames +
Datasets

Spark Streaming
(Structured
Streaming)

Machine Learning
MLlib

Graph
Processing
Graph X

Spark Core and Spark SQL Engine

Scala

SQL

Python

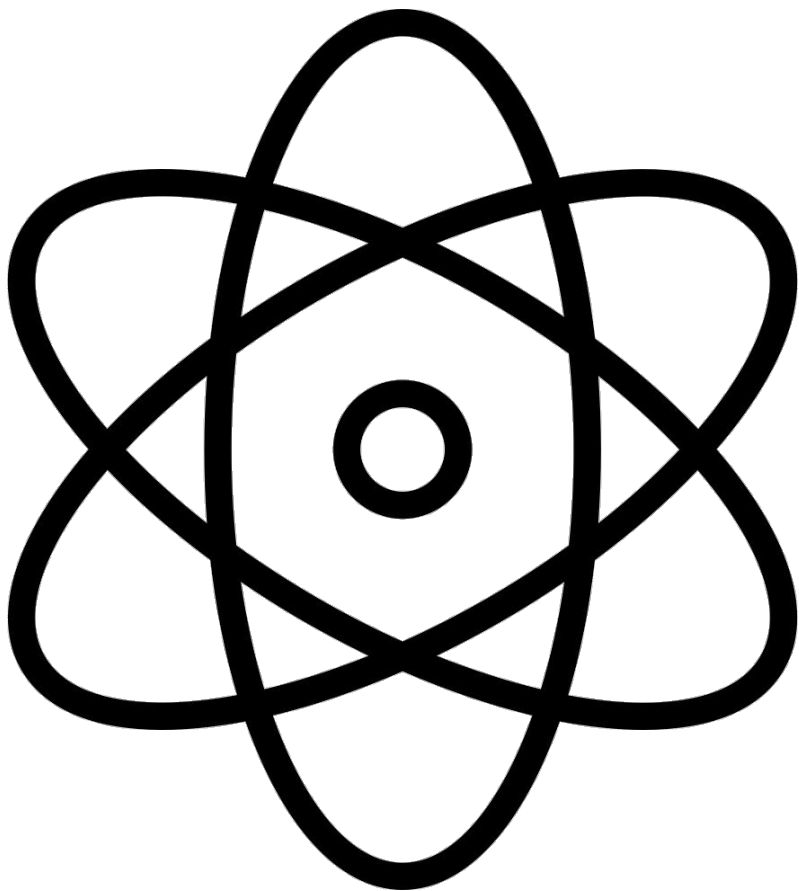
Java

R

Standalone Scheduler

YARN

KUBERNETES



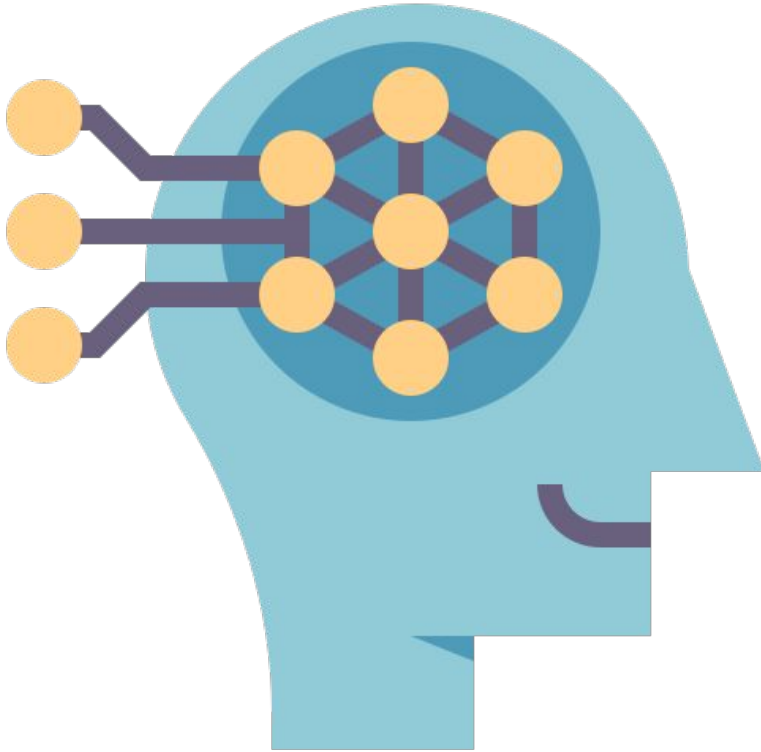
- Funcionalidade básicas do Spark
- Agendamento
- Gerenciamento de memória
- Recuperação de falhas
- Interação com sistemas de armazenamento
- Conjunto de dados
- APIs para manipulação de dados



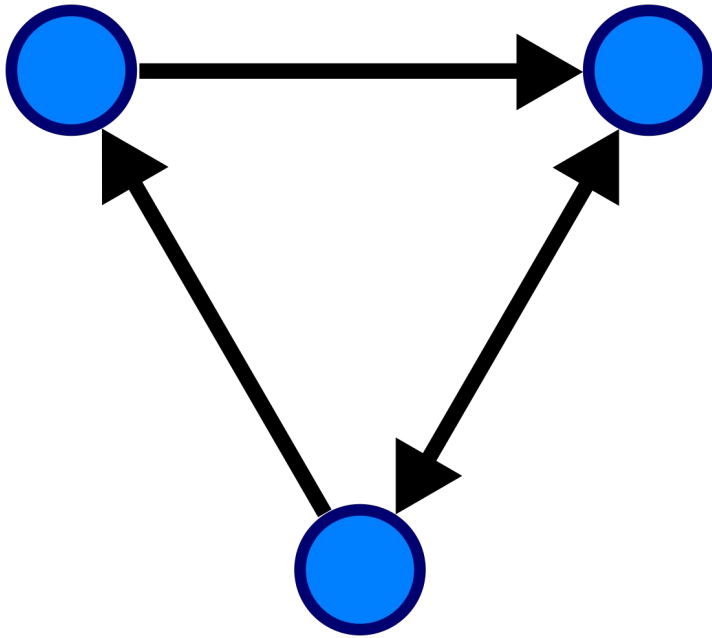
- Permite trabalhar com dados estruturados
- Consultas SQL, similar ao HQL
- Suporta várias fontes de dados como Hive, parquet, Json, RDDs
- Antigo projeto Shark da UC Berkeley
- Permite mesclar consultas SQL com manipulação de dados
- Suportado por Java, Scala e Python



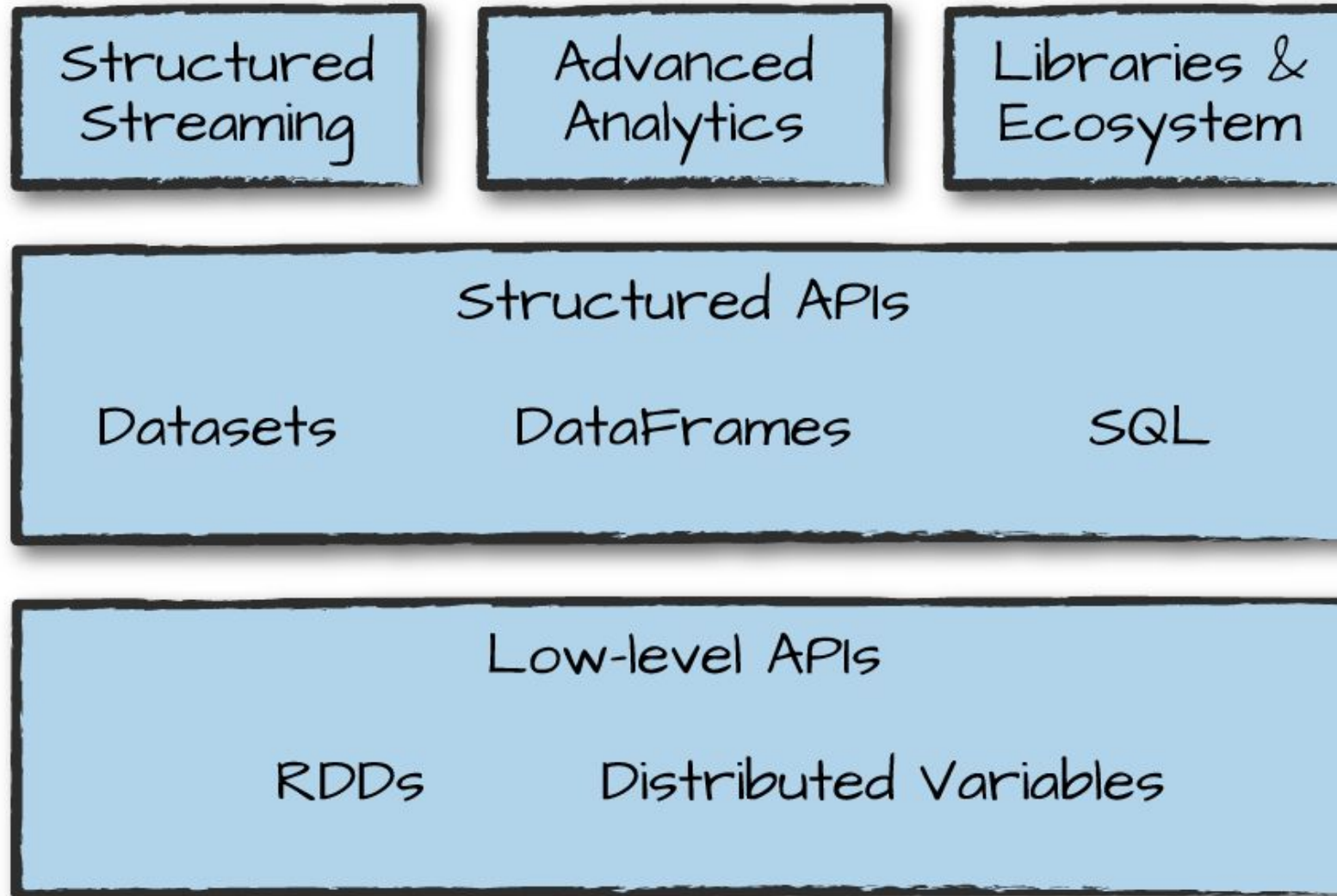
- Processamento de dados em Real Time
- Processamento de dados como logs de web services e mensagens de filas
- Escalável e tolerante a falhas
- Fluxo de dados similar ao DF do Spark Core
- Processamento em mini batch

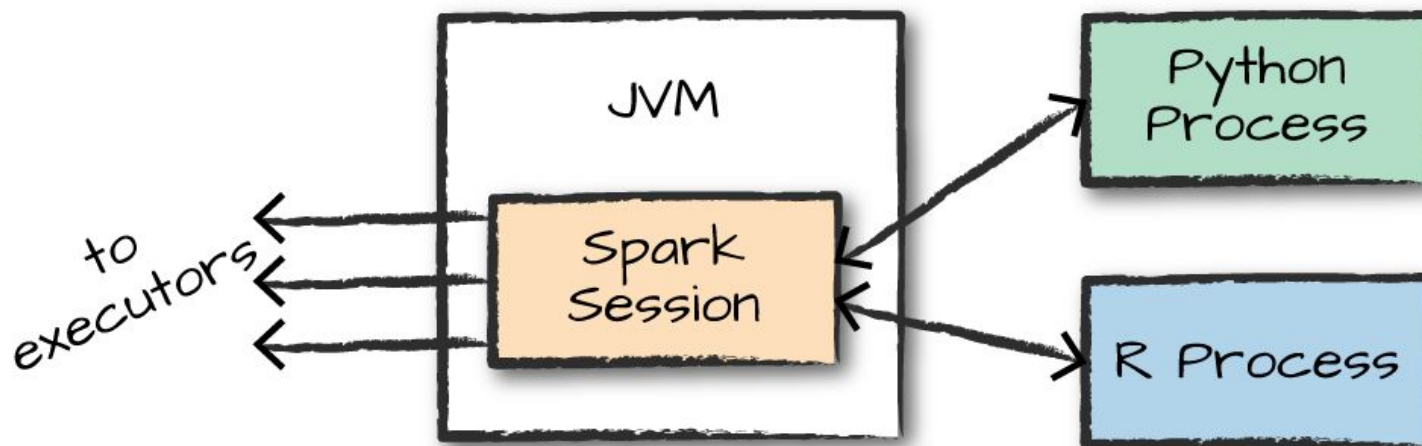


- Biblioteca de aprendizagem de máquina
- Vários tipos de algoritmos como classificação, regressão, filtragem colaborativa, etc..
- Todos métodos são projetados para escalar em um cluster



- Biblioteca para manipular grafos
- Vários operadores para manipular gráficos como subgrafo e mapVertices
- Permite criar um gráfico direcionado com propriedades arbitrárias anexadas a cada vértice e borda





Spark: The Definitive Guide, Bill Chambers, Matei Zaharia

Ao utilizar Spark com linguagem Python ou R, o Spark traduz o código para ser executado nas JVMs do executor.

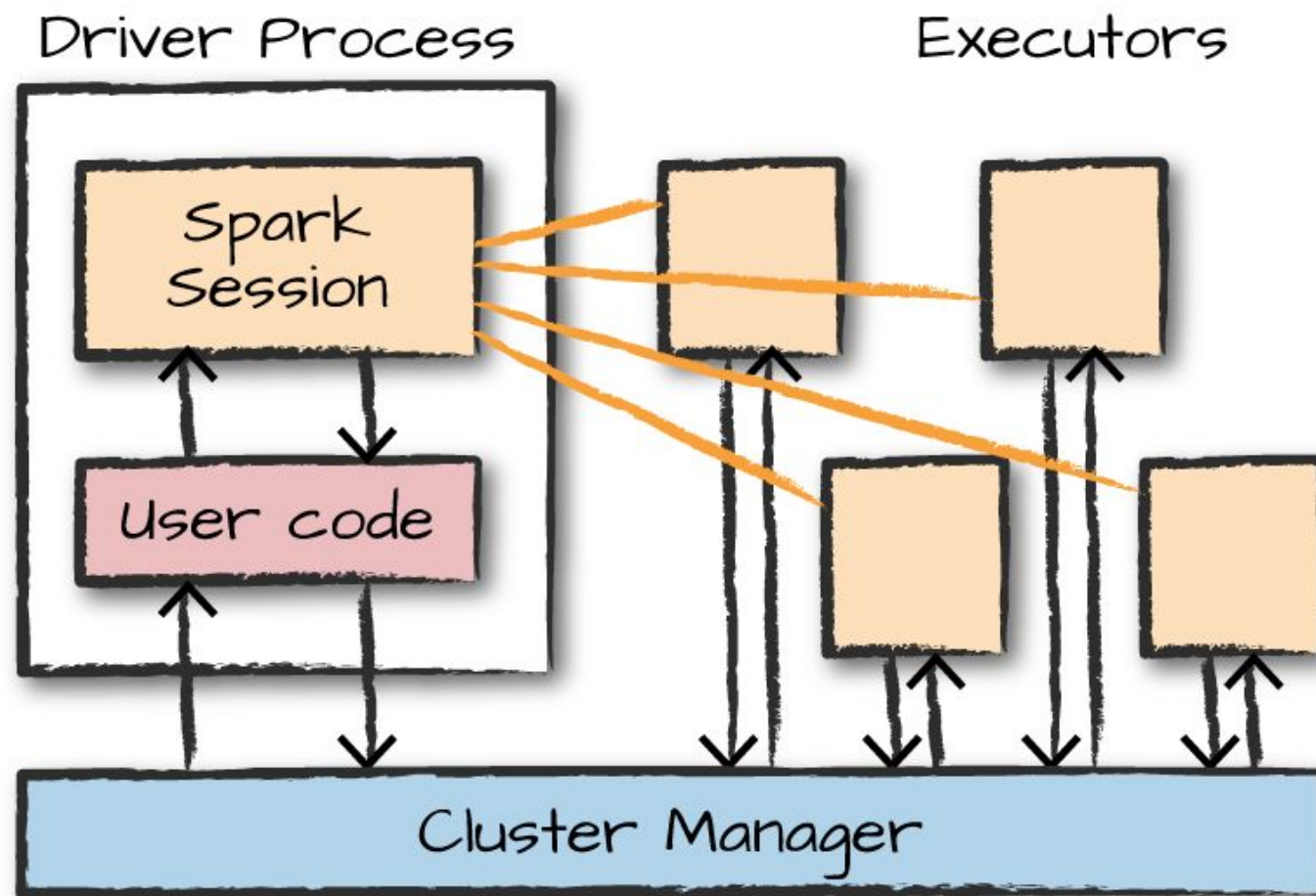
Ex: No PySpark, os códigos Python e JVM vivem em processos de sistema operacional separados. O PySpark usa o Py4J, que é uma estrutura que facilita a interoperação entre as duas linguagens, para trocar dados entre os processos Python e JVM.

Cluster Manager

Standalone – um gerenciador de cluster simples incluído no Spark que facilita a configuração de um cluster.

Hadoop YARN – o gerenciador de recursos no Hadoop 2 e 3.

Kubernetes – um sistema de código aberto para automatizar a implantação, dimensionamento e gerenciamento de aplicativos em contêineres.



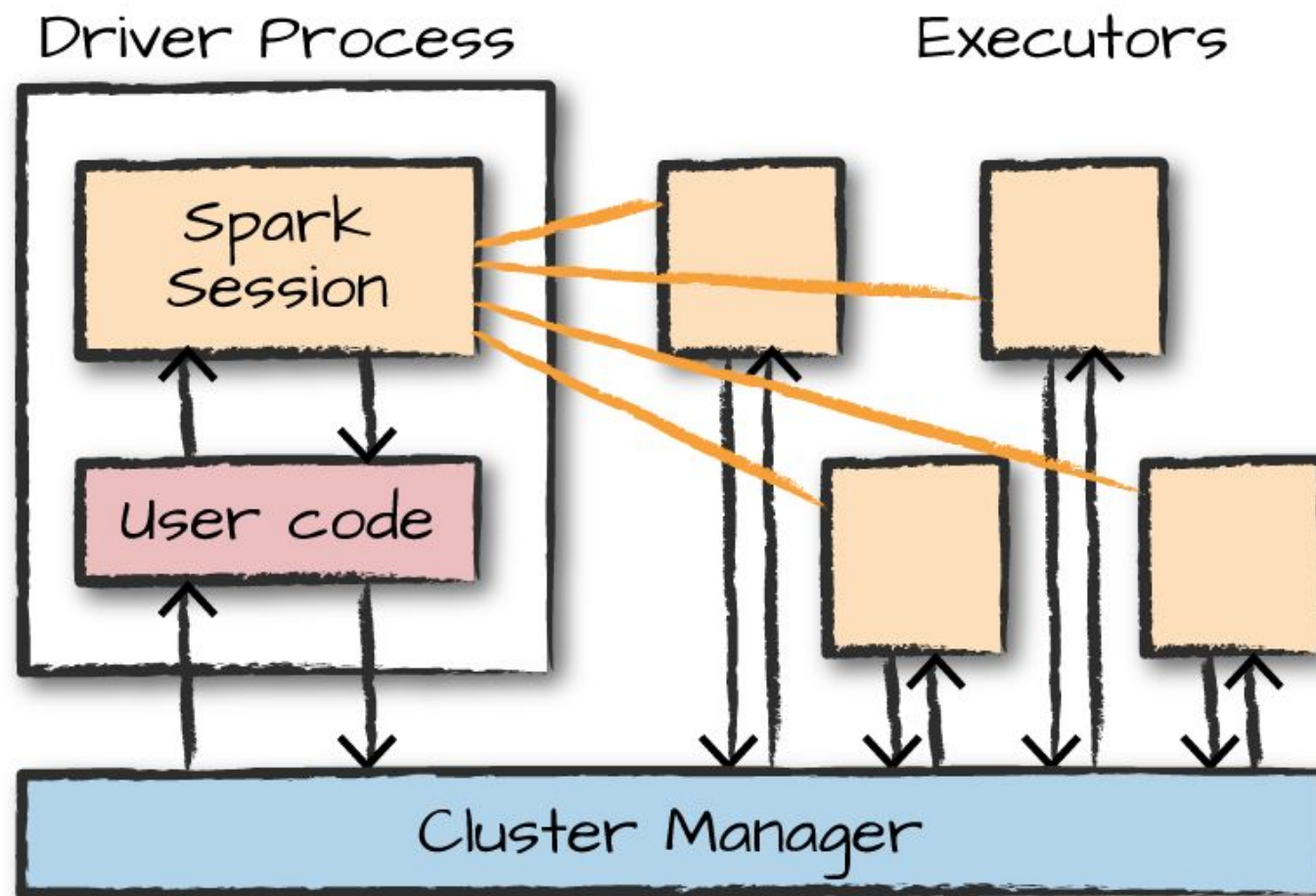
Execução em Cluster

Spark Session – Ponto inicial de um programa Spark.

Executors – Conjunto de cpu e memória onde são executados os programas.

Driver – Nó no cluster que o programa foi inicialmente executado.

Modo de execução – Local, Cluster e Cliente.



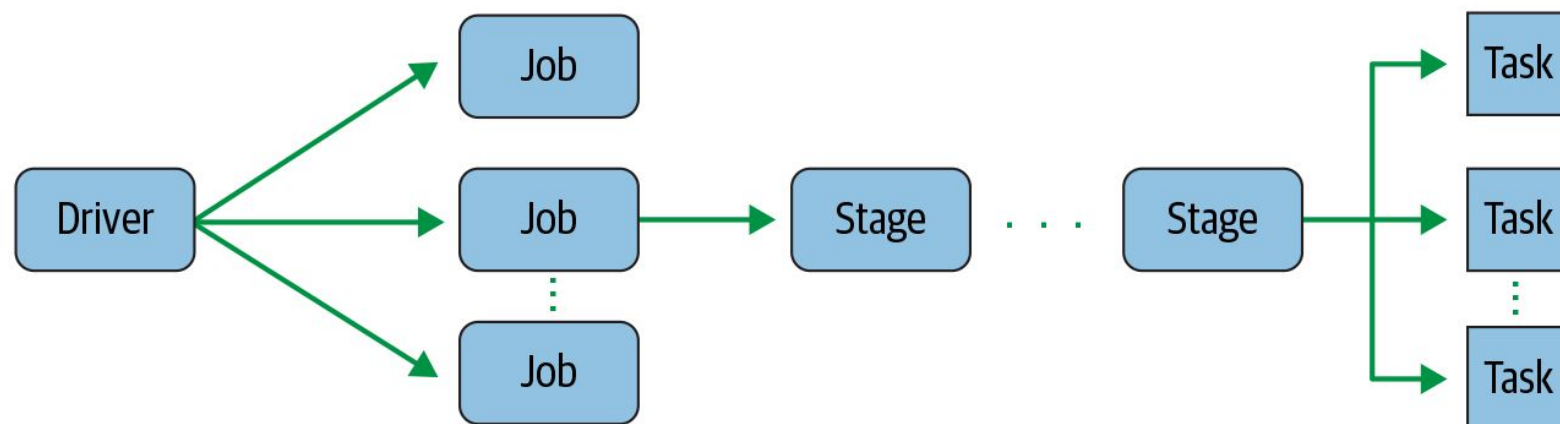
Application: Qualquer aplicação submetida no Spark

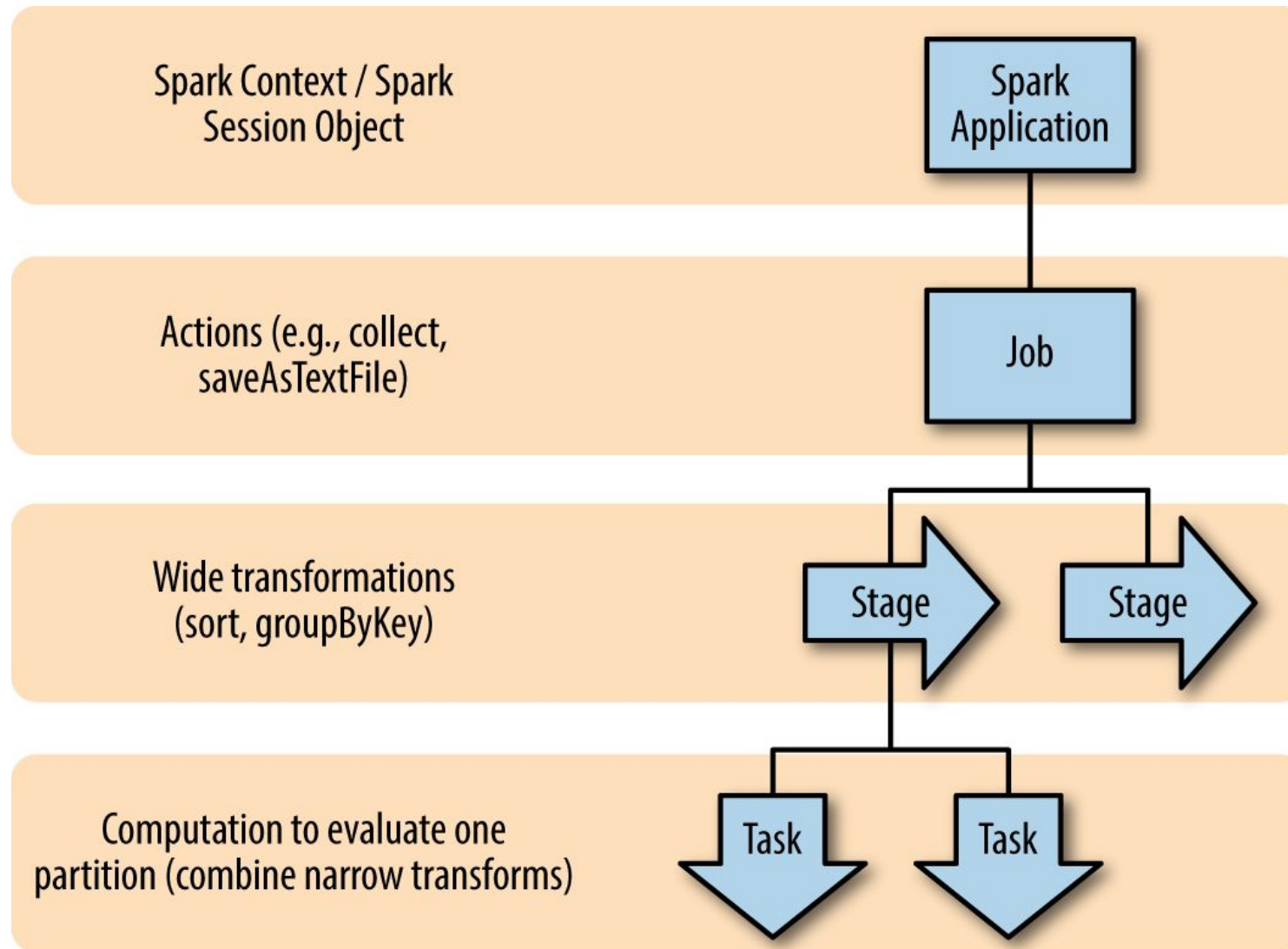
Job: Conjunto de transformações do que são geradas em resposta a uma ação submetida pela aplicação

Stage: Agrupamento de tasks que podem ser executadas independentes

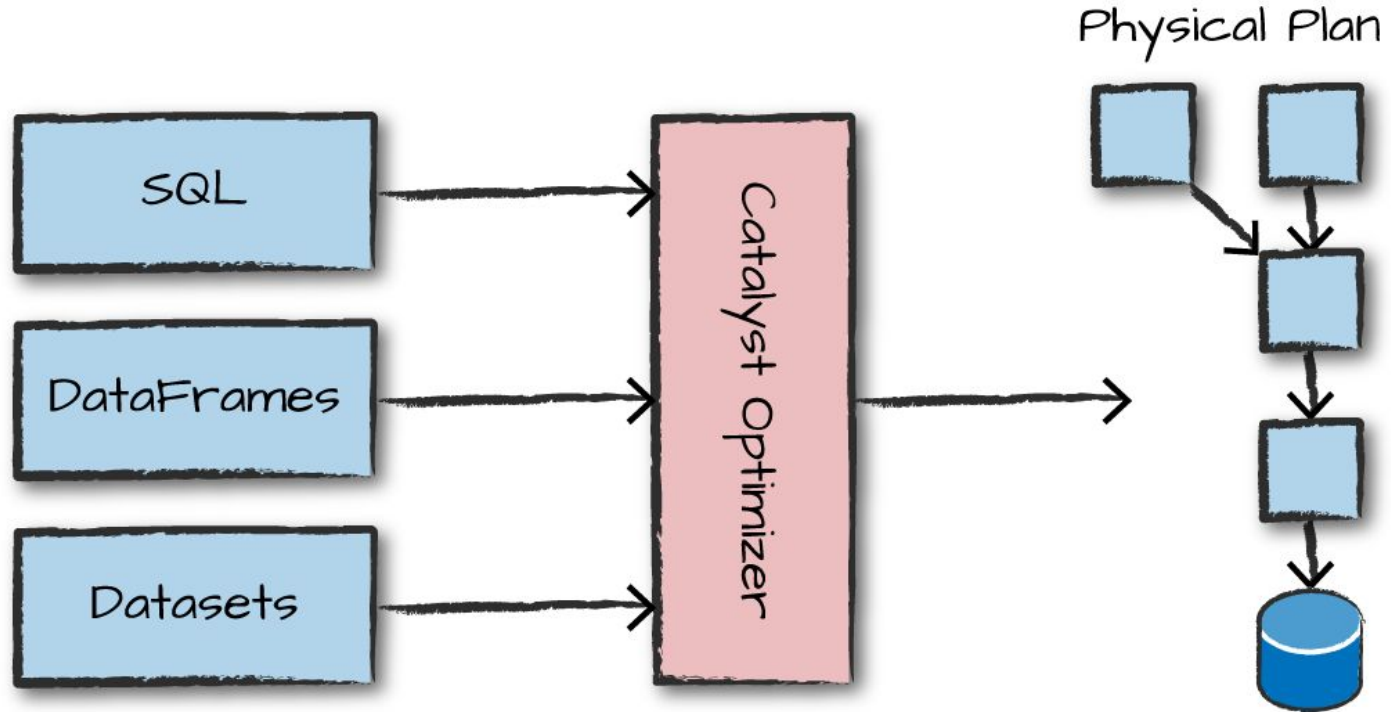
Task: Uma unidade de trabalho que será enviada para um executor

DAG: Grafo Acíclico Direcionado das dependências



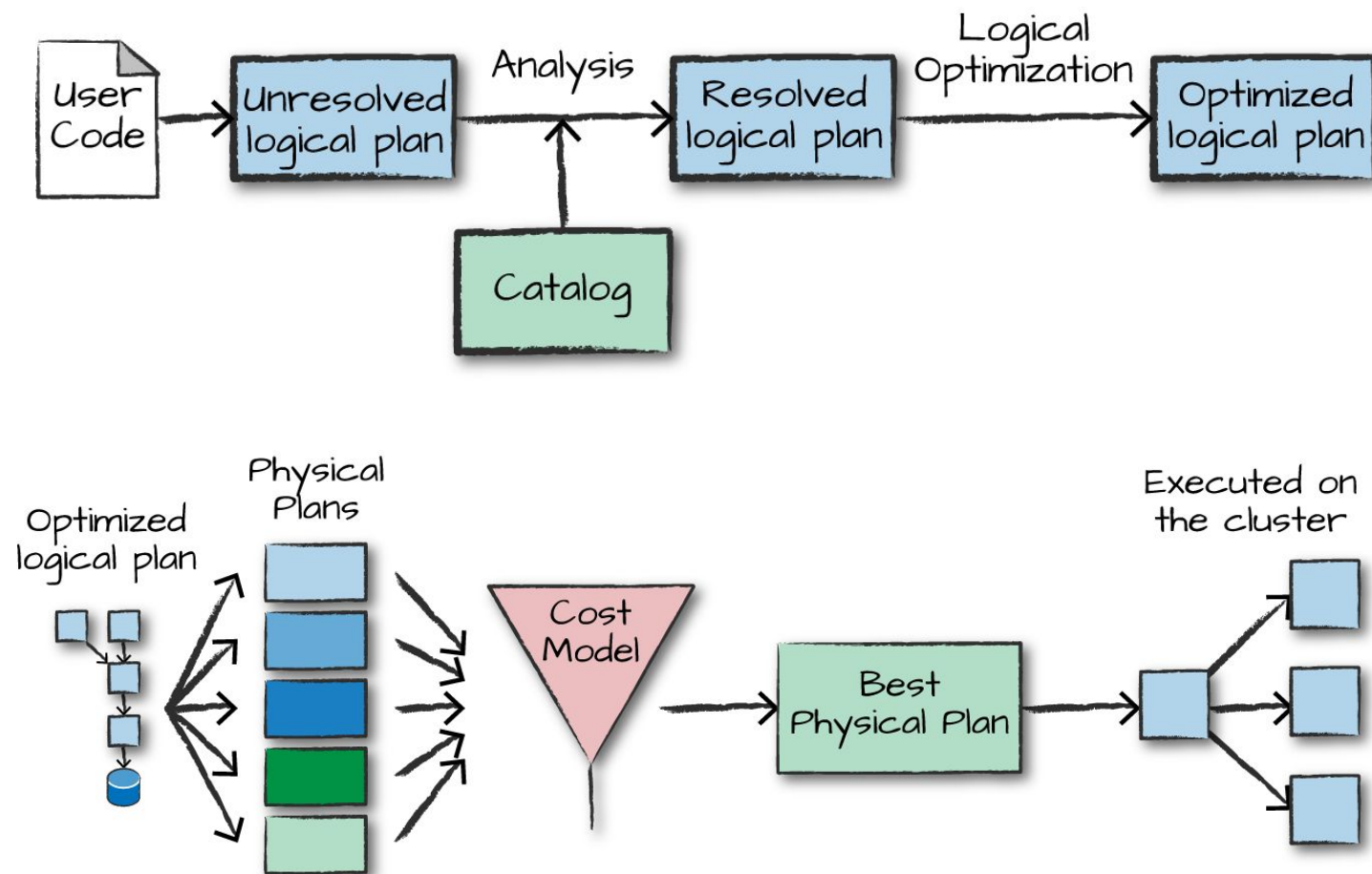


Catalyst Optimizer



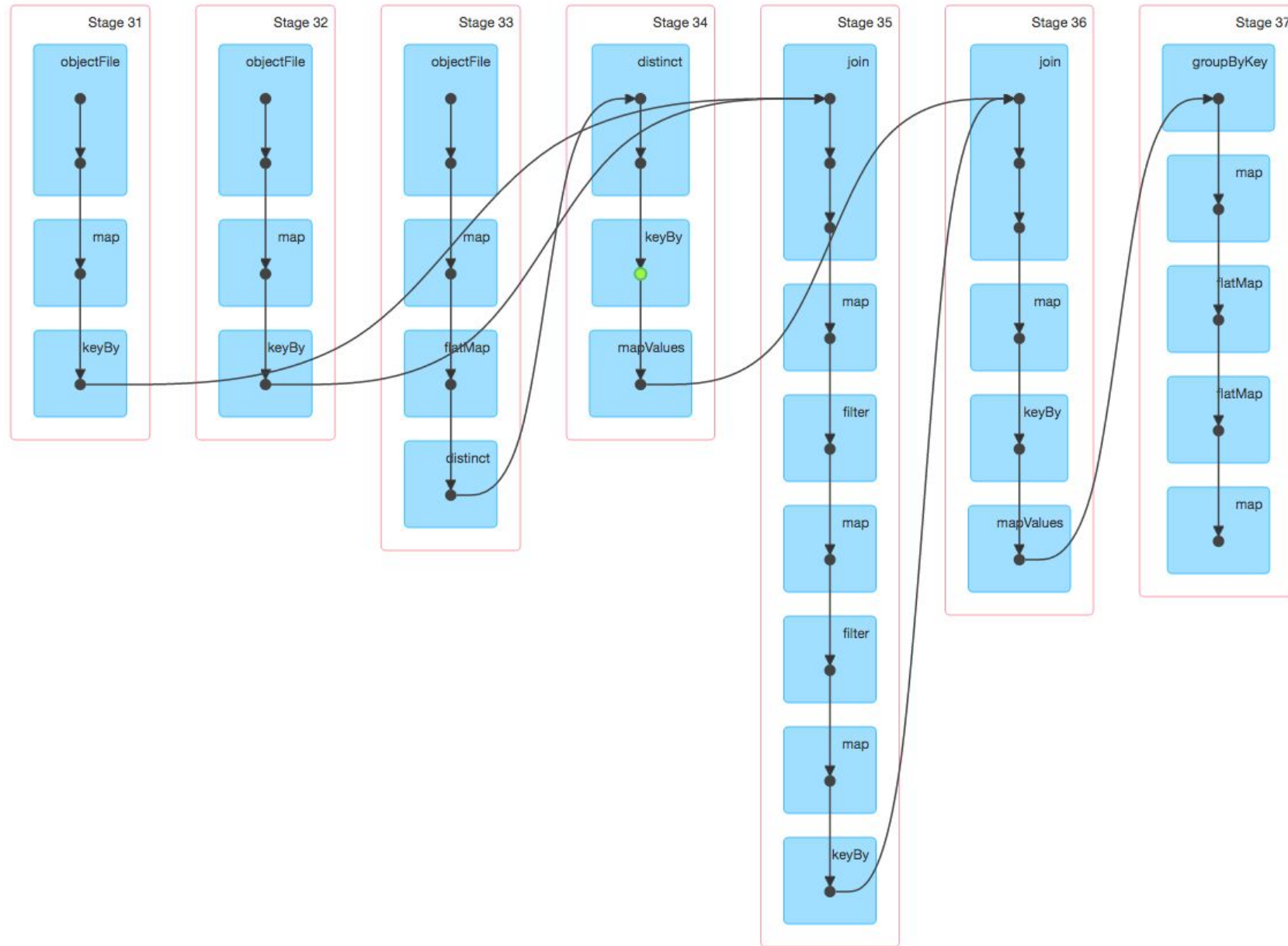
O código enviado ao Spark passa pelo Catalyst Optimizer, que decide como o código deve ser executado e estabelece um plano de execução, para que então o código seja executado e o resultado seja retornado ao usuário

Catalyst Optimizer



O plano lógico converter o conjunto de códigos em uma versão mais otimizada. O Spark usa um catálogo com as informações de tabela e DataFrame para resolver colunas e tabelas no analisador. Após essa validação resultado é enviado ao Catalyst Optimizer, que aplica regras que tentam otimizar o plano lógico. Depois de criar o plano lógico otimizado, o Spark inicia o processo de planejamento físico que representa como o plano lógico será executado no cluster

DAG



Spark UI

Spark 1.6.0 JobsStagesStorageEnvironmentExecutorsSQL PySparkShell application UI					
Spark Jobs (?) Total Uptime: 11.6 h Scheduling Mode: FIFO Completed Jobs: 119 Failed Jobs: 12 Event Timeline					
Completed Jobs (119)					
Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
130	showString at NativeMethodAccessorImpl.java:-2	2017/08/27 07:33:01	17 ms	1/1	1/1
129	showString at NativeMethodAccessorImpl.java:-2	2017/08/27 07:24:30	58 ms	1/1	1/1
128	showString at NativeMethodAccessorImpl.java:-2	2017/08/27 07:21:12	17 ms	1/1	1/1
127	showString at NativeMethodAccessorImpl.java:-2	2017/08/27 07:21:00	13 ms	1/1	1/1
126	showString at NativeMethodAccessorImpl.java:-2	2017/08/27 07:20:56	23 ms	1/1	1/1
125	showString at NativeMethodAccessorImpl.java:-2	2017/08/27 07:20:49	66 ms	1/1	1/1
124	count at null:-1	2017/08/27 07:15:46	54 ms	2/2	3/3
123	count at null:-1	2017/08/27 07:15:45	50 ms	2/2	3/3
122	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:45	55 ms	2/2	3/3
121	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:43	0.6 s	2/2	3/3
120	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:38	0.8 s	2/2	3/3
119	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:37	0.5 s	2/2	3/3
118	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:35	0.6 s	2/2	3/3
117	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:34	0.5 s	2/2	3/3
116	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:32	0.5 s	2/2	3/3
115	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:23	56 ms	2/2	3/3
114	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:21	69 ms	2/2	3/3
113	count at NativeMethodAccessorImpl.java:-2	2017/08/27 07:15:18	1 s	2/2	3/3

Cada SparkSession abre uma Web UI, por padrão na porta 4040, que exibe informações úteis sobre o aplicativo. Isso inclui:

- Uma lista de scheduler stage e tasks
- Um resumo do uso de memória
- Informação do ambiente
- Informações sobre executors

http://localhost:4040

- Resilient Distributed Dataset
- Uma coleção distribuída imutável de objetos tolerantes a falhas que podem ser operados em paralelo.
- Resilient (tolerância a falha) Distributed (distribuído no cluster) Dataset (conjunto de dados)
- Criados a partir de outro RDD ou de um conjunto externo de dados (HDFS, Hbase, Hive, etc)

```
lines = sc.textFile("data.txt")
lineLengths = lines.map(lambda s: len(s))
totalLength = lineLengths.reduce(lambda a, b: a + b)
```

```
counter = 0
rdd = sc.parallelize(data)

# Wrong: Don't do this!!
def increment_counter(x):
    global counter
    counter += x
rdd.foreach(increment_counter)

print("Counter value: ", counter)
```

- Semelhante ao RDD
- Representa uma coleção de dados distribuídos imutáveis, como um RDD
- Permite a imposição de estrutura a uma coleção distribuída de dados
- Possui um esquema, o que significa que é algo que pode ser visto como uma coluna com um nome e um tipo
- Pode ser manipulado com SQL

```
from datetime import datetime, date
from pyspark.sql import Row

df = spark.createDataFrame([
    Row(a=1, b=2., c='string1', d=date(2000, 1, 1), e=datetime(2000, 1, 1, 12, 0)),
    Row(a=2, b=3., c='string2', d=date(2000, 2, 1), e=datetime(2000, 1, 2, 12, 0)),
    Row(a=4, b=5., c='string3', d=date(2000, 3, 1), e=datetime(2000, 1, 3, 12, 0))
])
df.show()
```

	a	b	c	d	e
1	2.0	string1	2000-01-01	2000-01-01 12:00:00	
2	3.0	string2	2000-02-01	2000-01-02 12:00:00	
4	5.0	string3	2000-03-01	2000-01-03 12:00:00	

- Fornecer o melhor dos dois mundos (RDD e DataFrame)
- O estilo familiar de programação orientado a objetos e a segurança em tempo de compilação do RDD, mas com os benefícios de desempenho do otimizador de consulta do DataFrame
- Os conjuntos de dados também usam o mesmo mecanismo de armazenamento eficiente de pilha que o DataFrame.

```
// To create Dataset[Row] using SparkSession
```

```
val people = spark.read.parquet("...")
```

```
val department = spark.read.parquet("...")
```

```
people.filter("age > 30")
```

```
.join(department, people("deptId") === department("id"))
```

```
.groupBy(department("name"), people("gender"))
```

```
.agg(avg(people("salary")), max(people("age")))
```

```
// To create Dataset<Row> using SparkSession
```

```
Dataset<Row> people = spark.read().parquet("...");
```

```
Dataset<Row> department = spark.read().parquet("...");
```

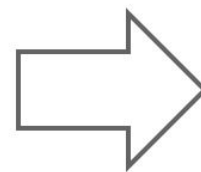
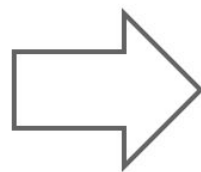
```
people.filter(people.col("age").gt(30))
```

```
.join(department, people.col("deptId").equalTo(department.col("id")))
```

```
.groupBy(department.col("name"), people.col("gender"))
```

```
.agg(avg(people.col("salary")), max(people.col("age")));
```


History of Spark APIs



Distribute collection
of JVM objects

Functional Operators (map,
filter, etc.)

Distribute collection
of Row objects

Expression-based operations
and UDFs

Logical plans and optimizer

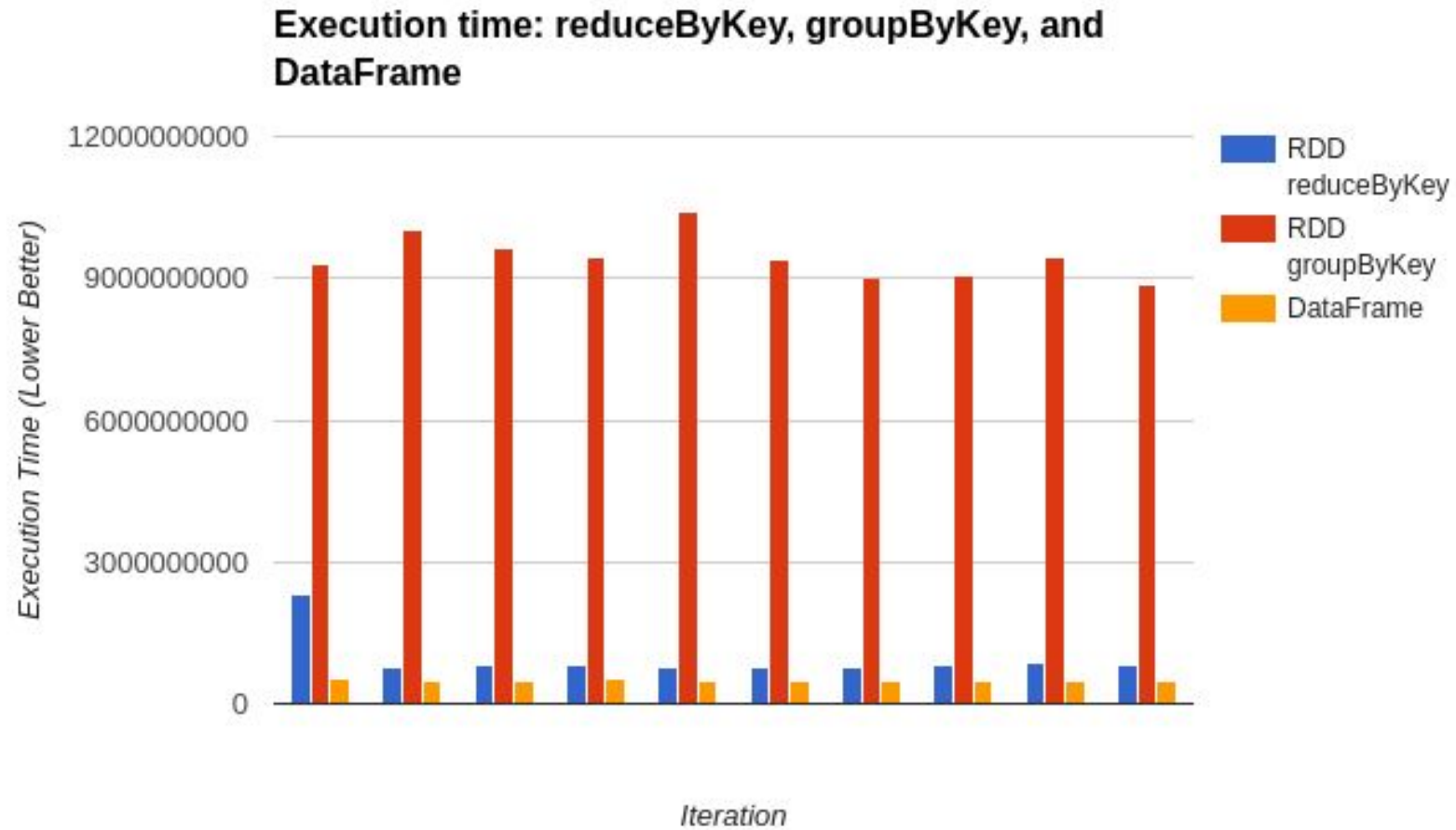
Fast/efficient internal
representations

Internally rows, externally
JVM objects

Almost the “Best of both
worlds”: type safe + fast

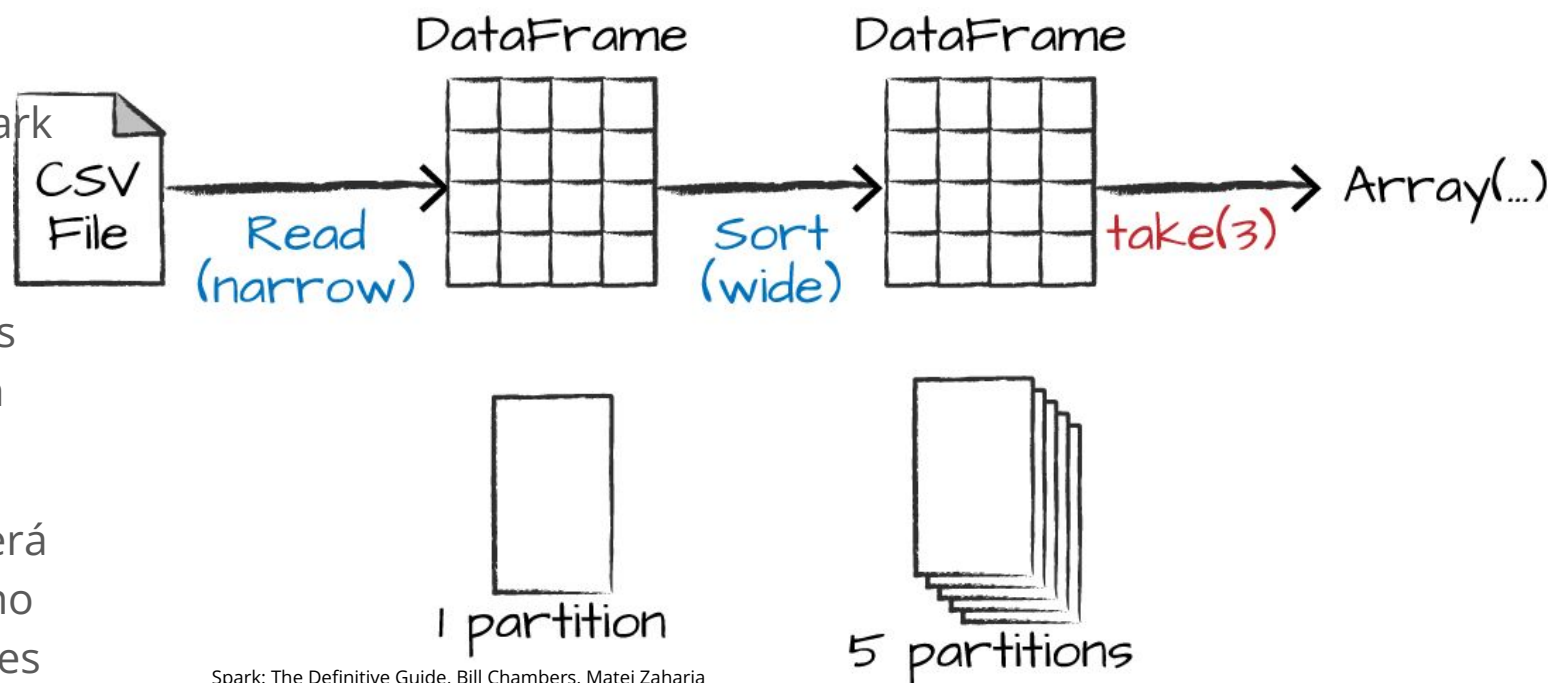
But slower than DF
Not as good for interactive
analysis, especially Python

RDD vs Dataframes





- Para permitir que cada executor execute o trabalho em paralelo, o Spark divide os dados em blocos chamados de partições
- Uma partição é uma coleção de linhas que ficam em uma máquina física em seu cluster
- Se você tiver uma partição, o Spark terá um paralelismo de apenas um, mesmo que você tenha milhares de executores
- Se você tiver muitas partições, mas apenas um executor, o Spark ainda terá um paralelismo de apenas um porque há apenas um recurso de computação.



Transformação

- Cria um DataFrame a partir de outro DataFrame

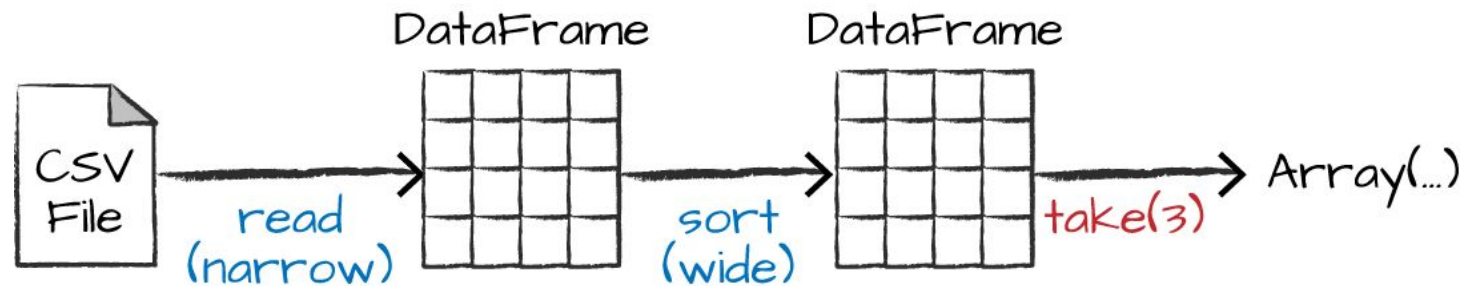
- Dois tipos de transformação: Wide e Narrow Dependencies

- Exemplos de transformações:

map() - Aplica uma função a cada elemento no RDD e retorna um RDD do resultado

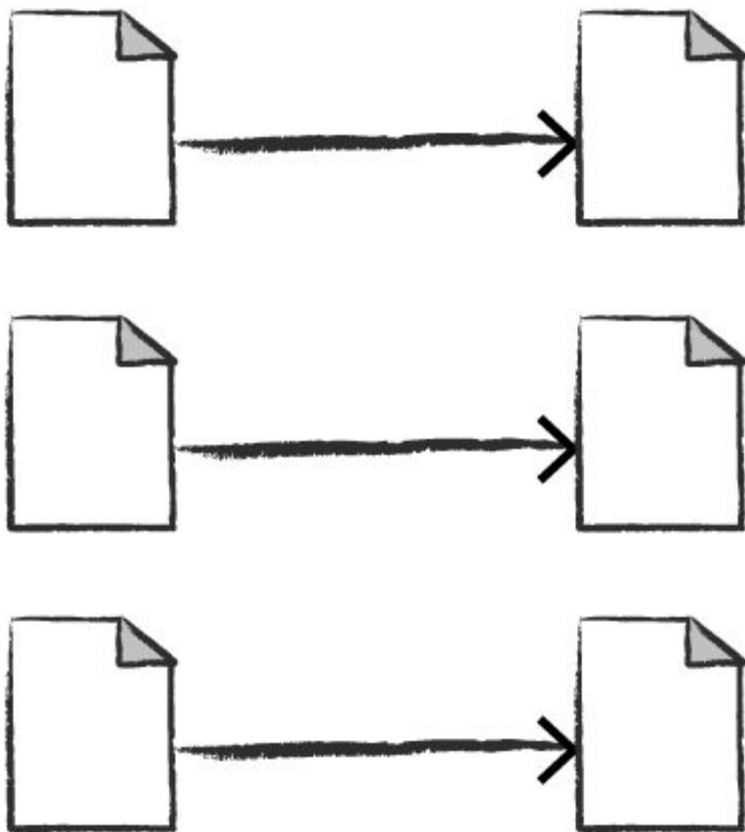
filter() - Retorna um RDD com os elementos que correspondem a condição de filtro

union() – Retorna um RDD contendo elementos de ambos os RDDs.



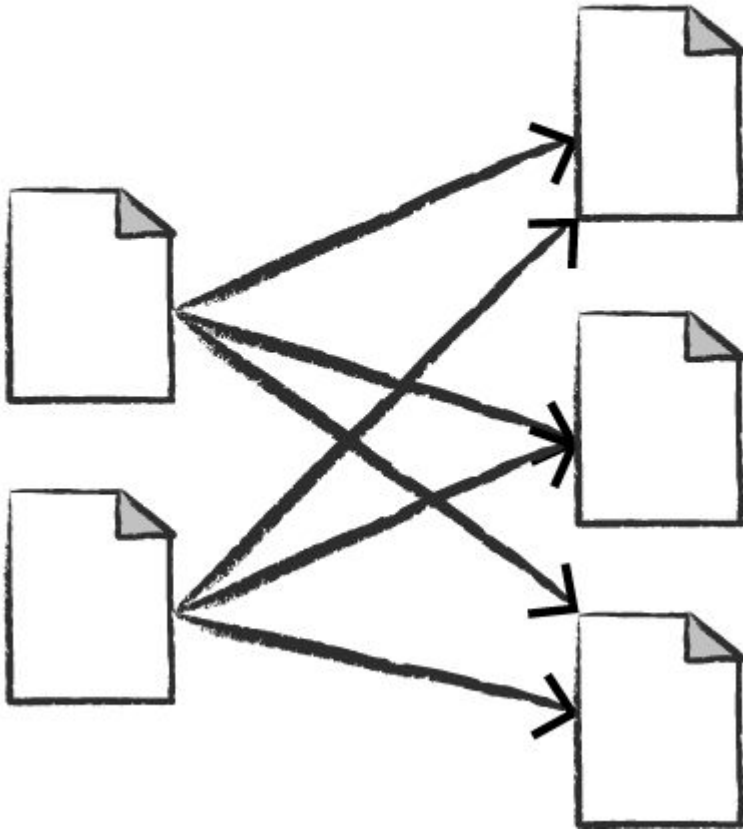
Spark: The Definitive Guide, Bill Chambers, Matei Zaharia

Narrow transformations 1 to 1



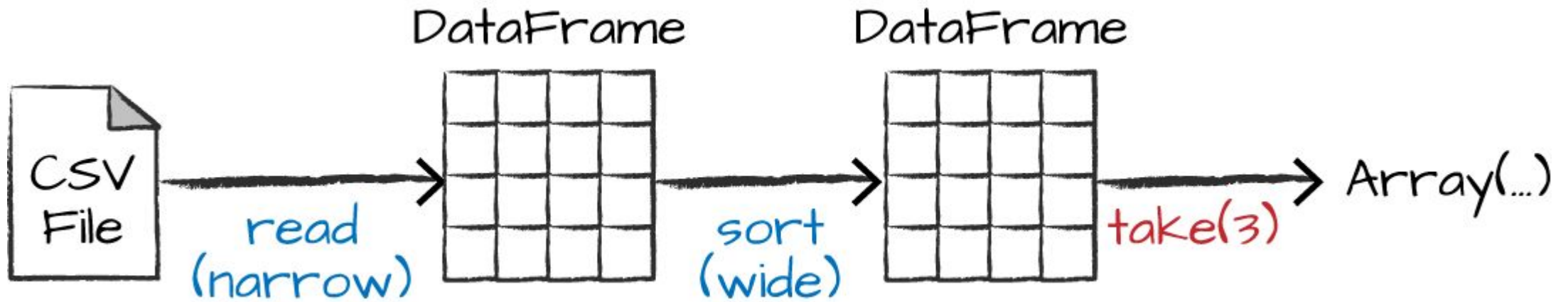
Dependências estreitas ou transformações estreitas são aquelas para as quais cada partição de entrada contribuirá para apenas uma partição de saída.

Wide transformations (shuffles) 1 to N



Dependência ampla ou transformação ampla terá partições de entrada contribuindo para muitas partições de saída, pode ser chamado também de shuffle, no qual o Spark trocará partições no cluster.

Example Narrow e wide



- Retorna o resultado ao driver ou salva em um sistema de armazenamento
- Exemplos de Ação:
 - `show()` - Retorna os dados na tela
 - `count()` - Retorna o número de elementos no DF
 - `head()` - Retorna a primeira linha do DF
 - `foreach(func)` - Aplica a função fornecida a cada elemento do RDD

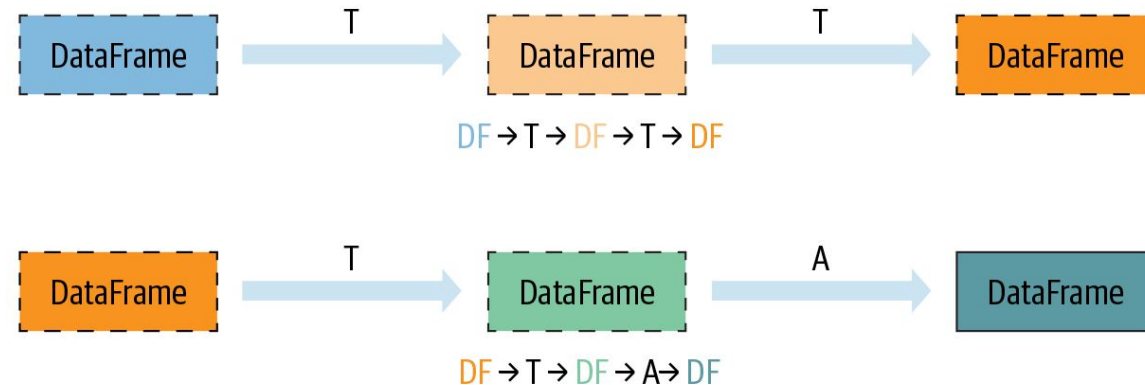
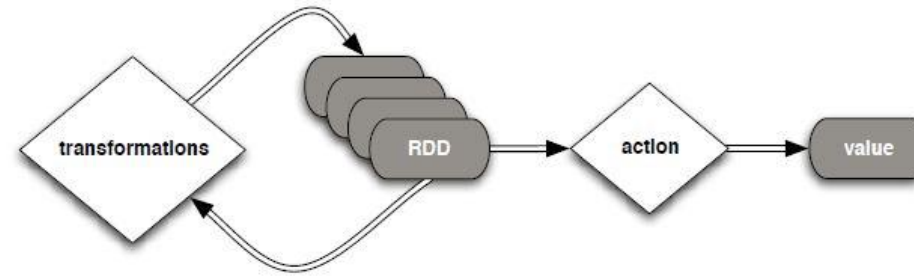
DataFrame



Spark: The Definitive Guide, Bill Chambers, Matei Zaharia

Lazy Evaluation

- As transformações não são executadas até que uma ação seja executada
- Spark internamente registra metadados para indicar que a operação foi solicitada
- Instruções sobre como calcular os dados que foram construídos através de transformações
- Utilizado para reduzir o número de passos, agrupando operações



T = Transformation A = Action

TRANSFORMAÇÃO T1

TRANSFORMAÇÃO T2

TRANSFORMAÇÃO T3

TRANSFORMAÇÃO A1

$T1 > T2 > T3$

ALUNO.CSV

DF_ALUNO

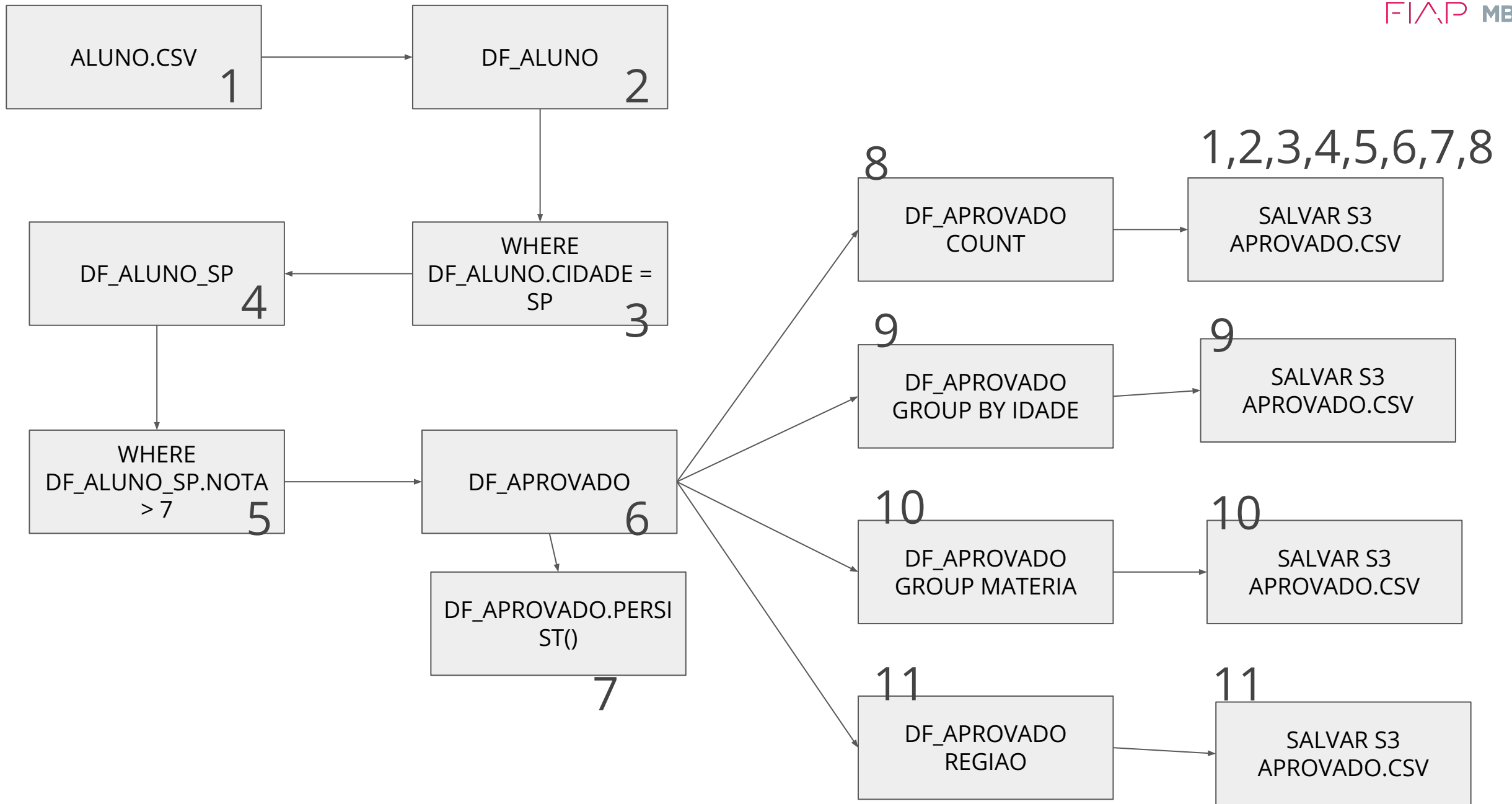
DF_ALUNO_SP

WHERE
DF_ALUNO.CIDADE =
SP

WHERE
DF_ALUNO_SP.NOTA
> 7

DF_APROVADO

SALVAR S3
APROVADO.CSV



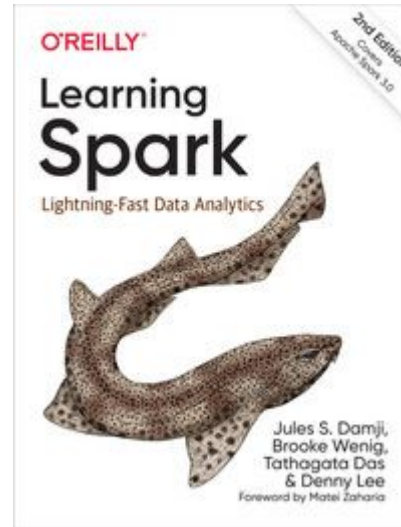
- Para evitar a computação de um DF várias vezes, podemos persistir os dados
- Os nós que compõem o DF armazenam suas partições
- Política de cache Least Used (LRU)
- Para adicionar o DF em cache é utilizado o método `persist()` e para retirar do cache `unpersist()`
- Também por ser utilizado o método `cache()`

Level	Space used	CPU time	In memory	On disk	Comments
MEMORY_ONLY	High	Low	Y	N	
MEMORY_ONLY_SER	Low	High	Y	N	
MEMORY_AND_DISK	High	Medium	Some	Some	Spills to disk if there is too much data to fit in memory.
MEMORY_AND_DISK_SER	Low	High	Some	Some	Spills to disk if there is too much data to fit in memory. Stores serialized representation in memory.
DISK_ONLY	Low	High	N	Y	

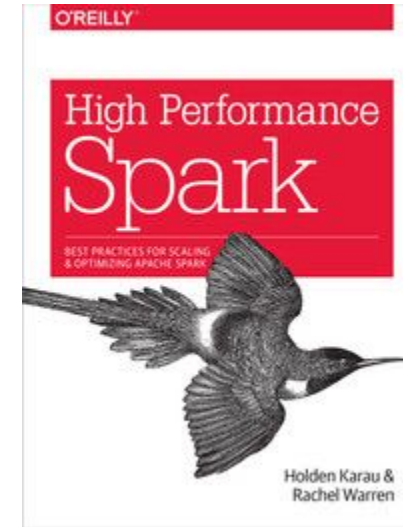
Referência Bibliográfica



Spark: The Definitive Guide
Bill Chambers, Matei Zaharia
Published by O'Reilly Media, Inc.
Apache Spark



Learning Spark, 2nd Edition
Jules S. Damji, Brooke Wenig, Tathagata Das, Denny Lee
Published by O'Reilly Media, Inc.
Apache Spark



High Performance Spark
Holden Karau, Rachel Warren
Published by O'Reilly Media, Inc.
Apache Spark

Site Oficial Apache Spark: <https://spark.apache.org/docs/latest/>

Site Oficial Databricks: <https://docs.databricks.com/lakehouse/index.html>